LAPORAN PRAKTIKUM 3

Struktur Data

RAFKI AHMAD PAGAMANDA

2311533016

Kelas D

Implementasi Struktur Data Stack

dan Penerapannya dalam Pemrograman Java

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

Implementasi Struktur Data Stack
dan Penerapannya dalam Pemrograman Java

A. Tujuan

Tujuan dari praktikum ini adalah utk memahami konsep struktur data stack dalam pemrograman Java. Pd praktikum ini, sy mempelajari bagaimana cara menyimpan dan mengelola data menggunakan prinsip LIFO (Last In First Out). Selain itu, sy jg belajar mengimplementasikan stack menggunakan array serta menguji beberapa operasi dasar seperti push, pop, dan peek. Dgn praktikum ini, sy diharapkan dapat memahami penerapan stack dalam kasus tertentu seperti pengolahan data dan perhitungan postfix.

B. Teori Singkat

Pd praktikum ini, sy mempelajari struktur data stack yg merupakan salah satu struktur data linear. Stack bekerja dgn prinsip LIFO (Last In First Out), dimana data yg terakhir masuk akan menjadi data pertama yg keluar. Konsep ini bisa diibaratkan seperti tumpukan barang, dimana elemen yg berada di atas akan diambil terlebih dahulu dibanding yg di bawah.

Dalam implementasinya, stack memiliki beberapa operasi utama seperti push utk menambahkan data ke dalam stack, pop utk menghapus data paling atas, dan peek utk melihat elemen teratas tanpa menghapusnya. Selain itu, terdapat jg kondisi seperti overflow ketika stack penuh dan underflow ketika stack kosong. Pd praktikum ini, sy menggunakan array sbg media penyimpanan stack, sehingga perlu mengatur posisi data menggunakan index agar operasi dapat berjalan dgn benar.

Selain menggunakan array, sebenarnya stack jg bisa diimplementasikan menggunakan struktur lain seperti ArrayList atau LinkedList. Namun pd praktikum ini, fokus utama sy adalah memahami konsep dasar terlebih dahulu menggunakan array agar lebih paham alur kerjanya.

Stack sendiri memiliki banyak penerapan dalam dunia pemrograman, seperti dalam proses pemanggilan fungsi (call stack), pengecekan keseimbangan tanda kurung, serta evaluasi ekspresi matematika seperti postfix.

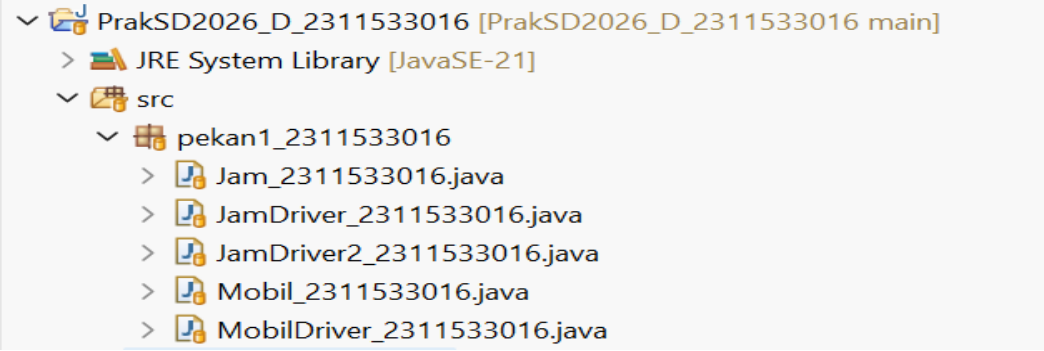
C. Alat dan Bahan

Pd praktikum ini, sy menggunakan laptop atau komputer yg sudah terinstall Eclipse IDE sbg media utama utk menulis dan menjalankan program. Bahasa pemrograman yg digunakan adalah Java karena praktikum ini masih berfokus pd konsep struktur data dan OOP.

Selain itu, bahan yg digunakan berupa source code yg dibuat selama praktikum, seperti class StackArray, StackPostfix, SiswaStack, dan class driver lainnya yg digunakan utk menjalankan program. Seluruh proses dilakukan langsung di dalam IDE, mulai dari penulisan kode hingga pengujian program.

D. Langkah-langkah

1. Pembuatan Package dan Class



Pd langkah ini, sy membuat package baru dgn nama pekan3_2311533016 di dalam project yg ada di Eclipse IDE. Package ini digunakan utk mengelompokkan seluruh file praktikum pekan 3 agar lebih rapi dan mudah dikelola. Setelah itu, sy membuat beberapa class di dalam package tersebut, yaitu StackArray, StackArrayDriver, StackPostfix, SiswaStack, dan NilaiMaksimum. Masing-masing class memiliki fungsi yg berbeda, dimana ada yg digunakan utk implementasi struktur data stack, ada yg digunakan utk menjalankan program, serta ada jg yg digunakan utk penerapan stack pada kasus tertentu. Dgn adanya pembagian ini, program menjadi lebih terstruktur dan mudah dipahami.

2. Pembuatan Class StackArray

```java
package pekan3_2311533016;

public class StackArray_2311533016 {

    static final int MAX = 1000;
    int top;
    int a[] = new int[MAX];

    boolean isEmpty() {
        return (top < 0);
    }

    StackArray_2311533016() {
        top = -1;
    }

    boolean push(int x) {
        if (top >= (MAX - 1)) {
            System.out.println("Stack Overflow");
            return false;
        } else {
            a[++top] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
        }
    }

    int pop() {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        } else {
            int x = a[top--];
            return x;
        }
    }

    int peek() {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        } else {
            int x = a[top];
            return x;
        }
    }

    void print() {
        for (int i = top; i > -1; i--) {
            System.out.print(" " + a[i]);
        }
```

Pd langkah ini, sy membuat class StackArray_2311533016 yg digunakan utk mengimplementasikan struktur data stack menggunakan array. Di dalam class ini, sy mendefinisikan array a[] sbg tempat penyimpanan data serta variabel top utk menunjukkan posisi elemen teratas pada stack. Selain itu, terdapat konstanta MAX yg digunakan utk menentukan kapasitas maksimum stack.

Selanjutnya, sy mengimplementasikan beberapa method utama seperti push utk menambahkan data ke dalam stack, dimana akan dilakukan pengecekan apakah stack sudah penuh (overflow) atau belum. Kemudian terdapat method pop utk menghapus elemen teratas serta peek utk melihat elemen paling atas tanpa menghapusnya, dimana keduanya jg melakukan pengecekan kondisi stack kosong (underflow). Selain itu, sy menambahkan method isEmpty utk mengecek apakah stack kosong, serta method print utk menampilkan isi stack dari atas ke bawah. Dgn adanya class ini, sy bisa memahami bagaimana cara kerja stack secara manual menggunakan array.

3. Pembuatan Class StackArrayDriver

```java
package pekan3_2311533016;

public class StackArrayDriver_2311533016 {

    public static void main(String[] args) {

        StackArray_2311533016 s = new StackArray_2311533016();

        s.push(10);
        s.push(20);
        s.push(30);

        System.out.println(s.pop() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah : " + s.peek());
        System.out.print("Elemen pada stack : ");

        s.print();
    }
}
```

Pd langkah ini, sy membuat class StackArrayDriver_2311533016 yg digunakan utk menguji class StackArray. Di dalam method main, sy membuat object stack yaitu s, kemudian menambahkan beberapa data ke dalam stack menggunakan method push, seperti nilai 10, 20, dan 30. Proses ini bertujuan utk melihat bagaimana data masuk ke dalam stack sesuai prinsip LIFO.

Setelah itu, sy mencoba mengambil satu data menggunakan method pop, sehingga dapat melihat elemen terakhir yg dimasukkan akan keluar terlebih dahulu. Selanjutnya, sy menggunakan method peek utk melihat elemen teratas tanpa menghapusnya. Di bagian akhir, sy menampilkan seluruh isi stack menggunakan method print agar dapat melihat urutan data yg masih tersisa di dalam stack.

Dgn adanya percobaan ini, sy jadi lebih memahami bagaimana perubahan data terjadi setiap kali operasi dilakukan pd stack. Sy bisa melihat secara langsung perbedaan kondisi sebelum dan sesudah dilakukan push dan pop, sehingga konsep stack tidak hanya dipahami secara teori, tetapi jg melalui praktik langsung.

4. Pembuatan Class StackPostfix

```java
package pekan3_2311533016;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2311533016 {

    public static int postfixEvaluate(String expression) {

        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner(expression);

        while (input.hasNext()) {
            if (input.hasNextInt()) { // operand
                s.push(input.nextInt());
            } else { // operator
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();

                if (operator.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);
                }
            }
        }

        input.close();
        return s.pop();
    }

    public static void main(String[] args) {
        System.out.println("hasil postfix = " + postfixEvaluate("5 2 4 * + 7 -"));
    }
}
```

Pd langkah ini, sy membuat class StackPostfix_2311533016 yg digunakan utk menghitung hasil dari ekspresi postfix menggunakan struktur data stack. Di dalam method postfixEvaluate, sy menggunakan class Stack utk menyimpan operand selama proses perhitungan berlangsung. Selain itu, sy jg menggunakan Scanner utk membaca setiap elemen dari ekspresi yg dimasukkan.

Proses perhitungan dilakukan dgn membaca satu per satu elemen dalam ekspresi. Jika elemen berupa angka, maka akan dimasukkan ke dalam stack menggunakan push. Namun jika elemen berupa operator, maka sy akan mengambil dua nilai teratas dari stack menggunakan pop, kemudian melakukan operasi sesuai operator seperti penjumlahan, pengurangan, perkalian, atau pembagian. Hasil dari operasi tersebut kemudian dimasukkan kembali ke dalam stack.

Setelah seluruh ekspresi selesai diproses, hasil akhir akan diambil dari stack dan ditampilkan. Pd bagian main, sy mencoba contoh ekspresi postfix utk melihat apakah hasil yg diperoleh sudah sesuai. Dgn langkah ini, sy bisa memahami bagaimana stack digunakan dalam kasus nyata seperti evaluasi ekspresi matematika.

Selain itu, sy jg memahami bahwa urutan pengambilan operand sangat berpengaruh terhadap hasil operasi, terutama pd operator seperti pengurangan dan pembagian. Oleh karena itu, penting utk memperhatikan urutan saat melakukan pop dari stack agar hasil perhitungan tidak salah.

5. Pembuatan Class SiswaStack

```java
package pekan3_2311533016;

import java.util.ArrayList;

class Siswa {
    String nama;
    int nim;

    public Siswa(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim: " + nim + ", Nama: " + nama;
    }
}

public class SiswaStack_2311533016 {
    private ArrayList<Siswa> stack;

    public SiswaStack_2311533016() {
        stack = new ArrayList<>();
    }

    public void push(Siswa mhs) {
        stack.add(mhs);
    }

    public Siswa pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswa peek() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }

    public static void main(String[] args) {
        SiswaStack_2311533016 studentStack = new SiswaStack_2311533016();

        Siswa mhs1 = new Siswa("Ali", 1);
        Siswa mhs2 = new Siswa("Boby", 2);
        Siswa mhs3 = new Siswa("Charles", 3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        // === SESUAI FOTO TERAKHIR ===
        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa();

        System.out.println("Siswa teratas " + studentStack.peek());
        System.out.println("Mengeluarkan siswa teratas dari stack: " + studentStack.pop());

        System.out.println("Daftar siswa setelah di pop:");
        studentStack.tampilkanSiswa();
    }
}
```

Pd langkah ini, sy membuat class SiswaStack_2311533016 yg digunakan utk mengimplementasikan stack dgn menyimpan object Siswa menggunakan ArrayList. Di dalamnya terdapat class Siswa yg memiliki atribut nama dan nim, serta method toString utk menampilkan data. Selanjutnya, sy membuat beberapa method utama seperti push utk menambahkan data siswa, pop utk menghapus data terakhir, peek utk melihat data teratas, serta isEmpty utk mengecek apakah stack kosong. Selain itu, terdapat method tampilkanSiswa utk menampilkan seluruh isi stack. Pd bagian main, sy membuat beberapa object siswa lalu memasukkannya ke dalam stack, kemudian menampilkan data, melihat elemen teratas, serta menghapus data dari stack. Dgn langkah ini, sy dapat memahami bagaimana stack dapat digunakan utk menyimpan object, tidak hanya data sederhana seperti integer.

6. Pembuatan Class NilaiMaksimum

```java
package pekan3_2311533016;

import java.util.Stack;

public class NilaiMaksimum_2311533016 {

    public static int max(Stack<Integer> s) {

        Stack<Integer> backup = new Stack<Integer>();

        int maxValue = s.pop();
        backup.push(maxValue);

        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue = Math.max(maxValue, next);
        }

        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }

        return maxValue;
    }

    public static void main(String[] args) {

        Stack<Integer> s = new Stack<Integer>();

        s.push(70);
        s.push(12);
        s.push(20);

        System.out.println("isi stack " + s);
        System.out.println("Stack Teratas " + s.peek());
        System.out.println("Nilai maksimum " + max(s));
    }
}
```

Pd langkah ini, sy membuat class NilaiMaksimum_2311533016 yg digunakan utk mencari nilai maksimum dari data yg disimpan di dalam stack. Di dalam method max, sy menggunakan stack tambahan (backup) utk menyimpan sementara data yg di-pop dari stack utama. Proses dilakukan dgn mengambil satu per satu elemen menggunakan pop, kemudian membandingkannya utk mendapatkan nilai terbesar menggunakan fungsi Math.max. Setelah semua elemen diperiksa, data dikembalikan lagi ke stack semula agar isi stack tidak hilang.

Pd bagian main, sy membuat object stack lalu menambahkan beberapa nilai seperti 70, 12, dan 20 menggunakan push. Setelah itu, sy menampilkan isi stack, elemen teratas menggunakan peek, serta hasil nilai maksimum yg diperoleh dari method max.

E. Kesimpulan

Dari praktikum pekan ini, sy jadi lebih memahami konsep struktur data stack beserta cara kerjanya yg menggunakan prinsip LIFO. Sy belajar bagaimana mengimplementasikan stack menggunakan array, serta menggunakan berbagai operasi seperti push, pop, dan peek utk mengelola data. Selain itu, sy jg memahami kondisi seperti overflow dan underflow yg perlu diperhatikan saat menggunakan stack.

Melalui beberapa percobaan yg dilakukan, seperti evaluasi postfix, pengelolaan data siswa, hingga pencarian nilai maksimum, sy jadi lebih paham bahwa stack tidak hanya digunakan utk penyimpanan data saja, tetapi jg bisa diterapkan pada berbagai kasus dalam pemrograman.